Ulster University

Debugging

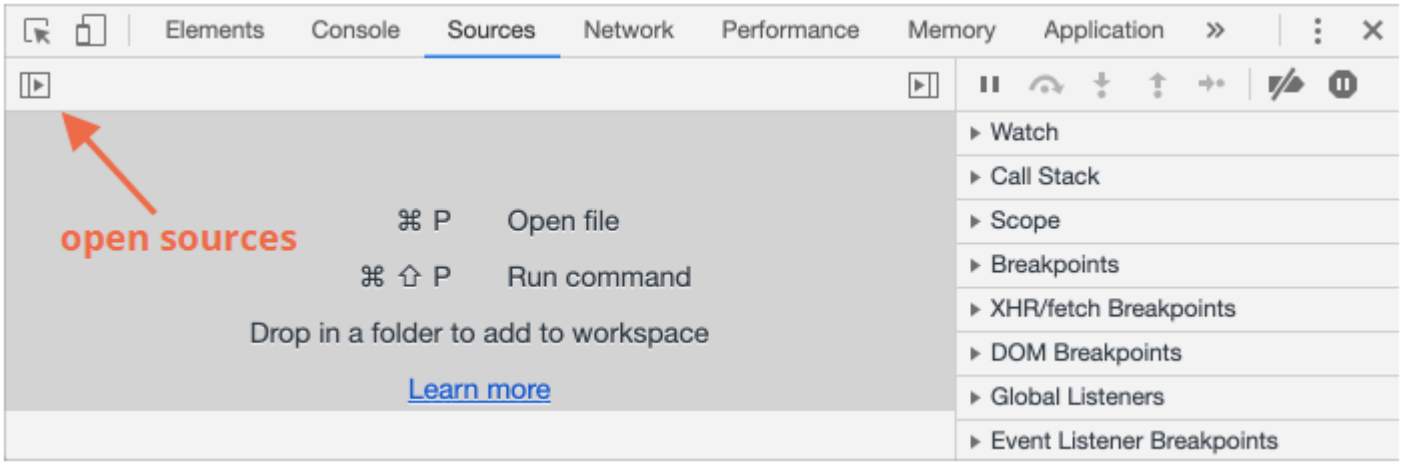
cp.shewell@ulster.ac.uk

What is debugging?

- Debugging is the process of finding and fixing errors within a script. All modern browsers and most other environments support debugging tools – a special UI in developer tools that makes debugging much easier.
- It also allows to trace the code step by step to see what exactly is going on.
- We'll be using Chrome here, because it has enough features, most other browsers have a similar process.

Sources Panel

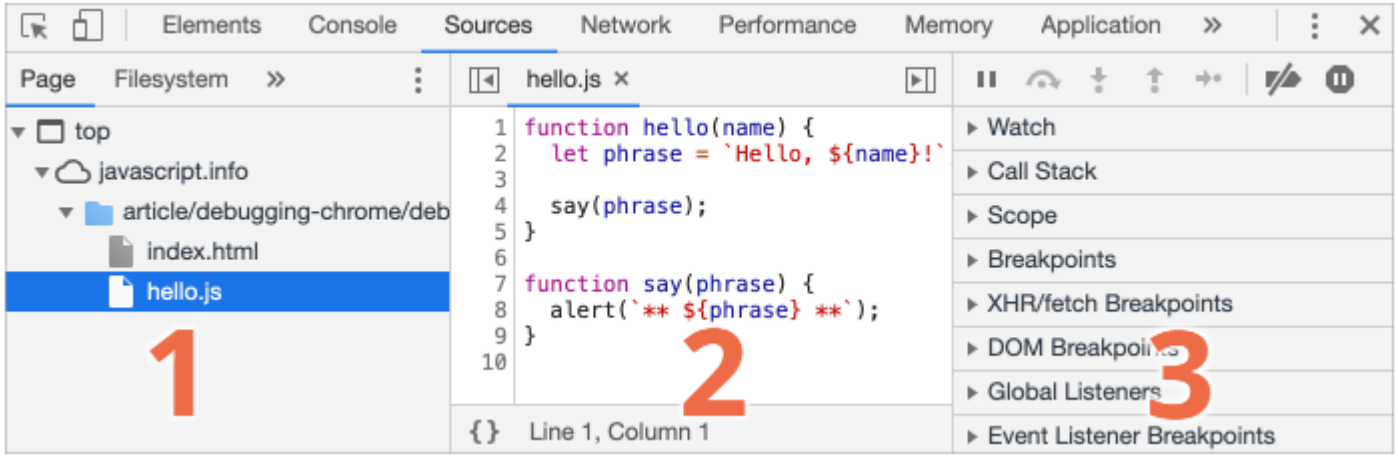
- Your Chrome version may look a little bit different, but it still should be obvious what's there.
- Here's what you should see if you are doing it for the first time:



- The toggler button opens the tab with files.

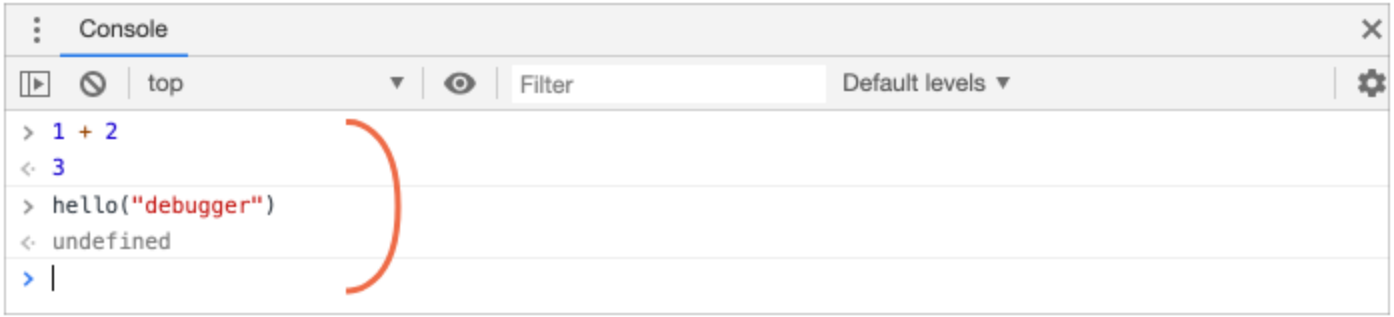
Sources Panel

- The Sources panel has 3 parts:
  1. The File Navigator pane lists HTML, JavaScript, CSS and other files, including images that are attached to the page. Chrome extensions may appear here too.
  2. The Code Editor pane shows the source code.
  3. The JavaScript Debugging pane is for debugging, we'll explore it soon.



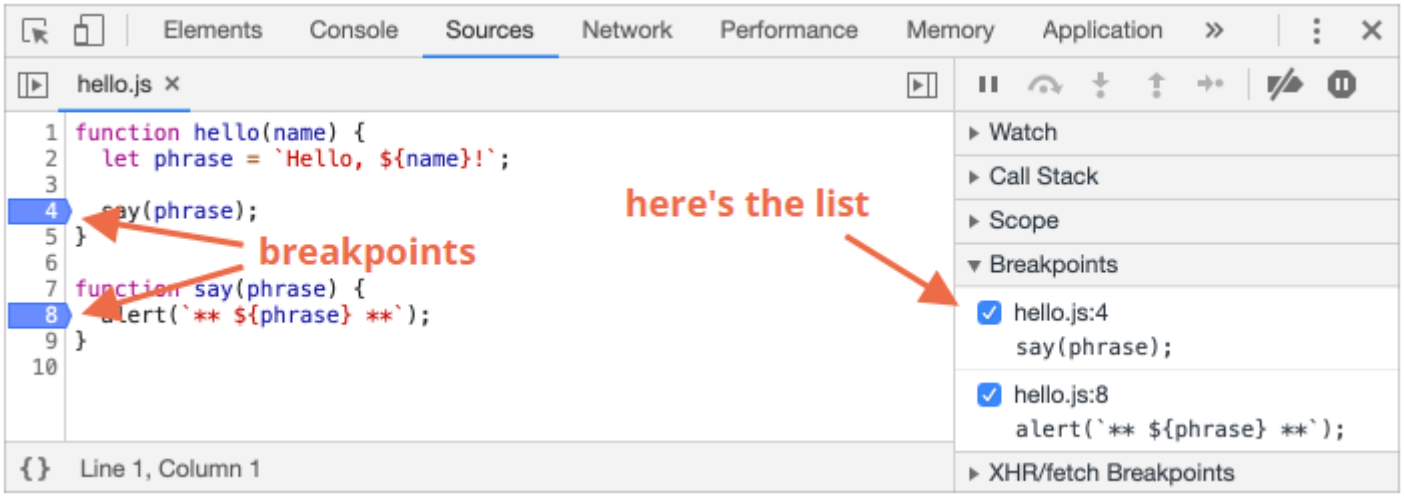
Console

- If we press Esc, then a console opens below. We can type commands there and press Enter to execute.
- After a statement is executed, its result is shown below.
- For example, here 1+2 results in 3, and hello("debugger") returns nothing, so the result is undefined:



Breakpoints

- A breakpoint is a point of code where the debugger will automatically pause the JavaScript execution.
- While the code is paused, we can examine current variables, execute commands in the console etc. In other words, we can debug it.
- We can always find a list of breakpoints in the right panel. That's useful when we have many breakpoints in various files.



Debugger Command

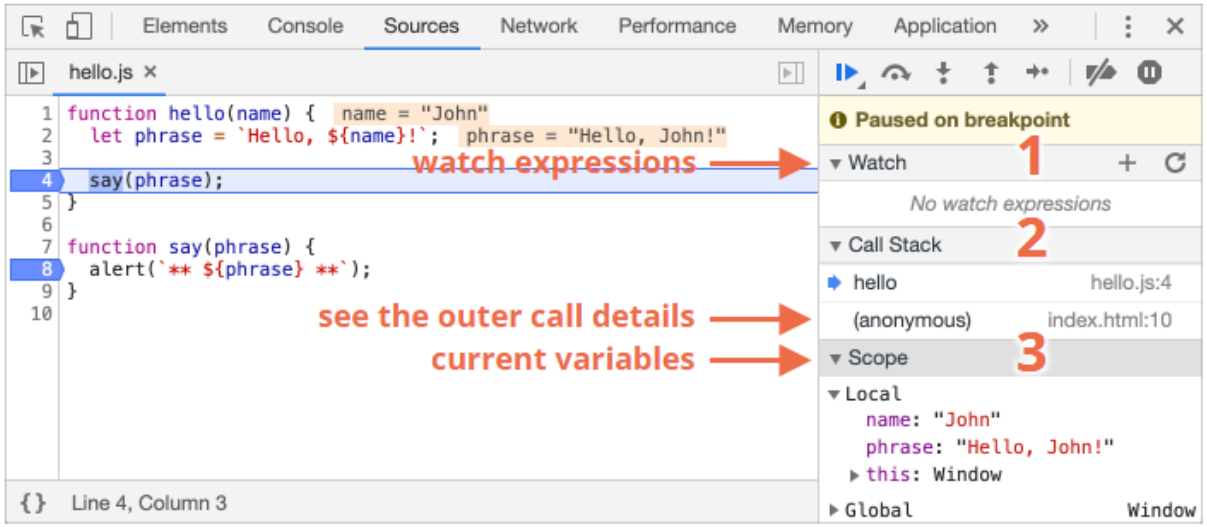
- We can also pause the code by using the debugger command in it, like this:

```
function hello(name) {
  let phrase = `Hello, ${name}!`;

  debugger;  // <-- the debugger stops here

  say(phrase);
}
```

- That's very convenient when we are in a code editor and don't want to switch to the browser and look up the script in developer tools to set the breakpoint.

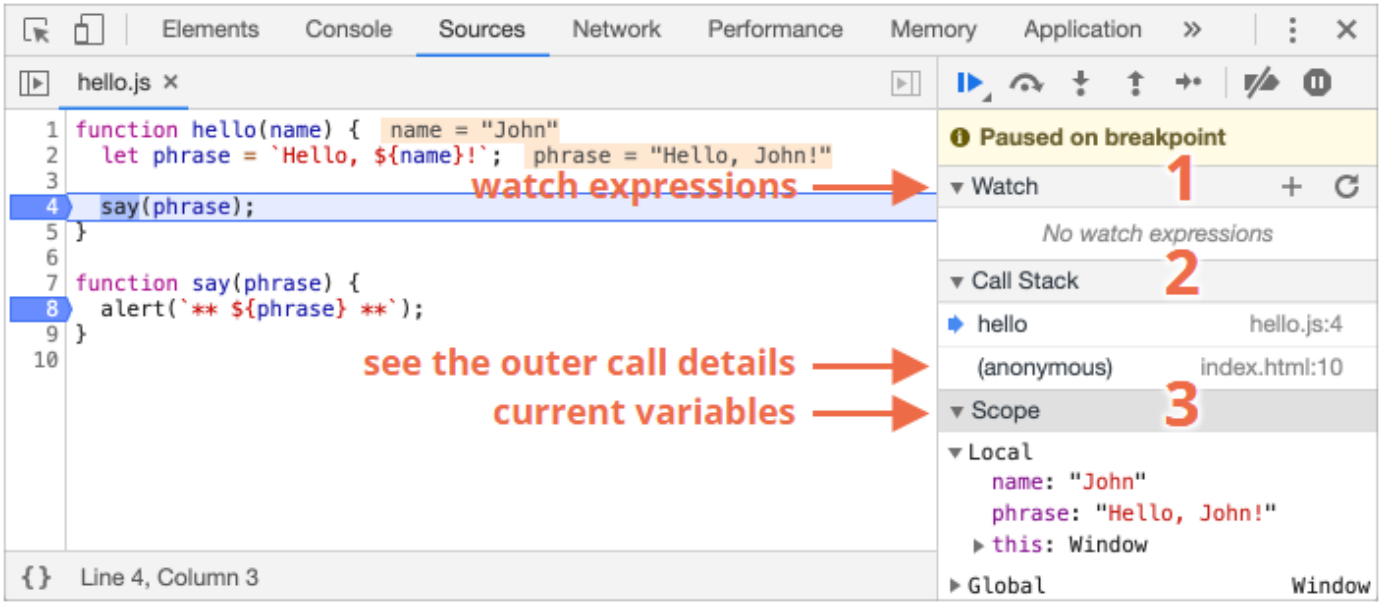
Pause and Look Around

- As the breakpoint is set, the execution pauses at the 4th line:



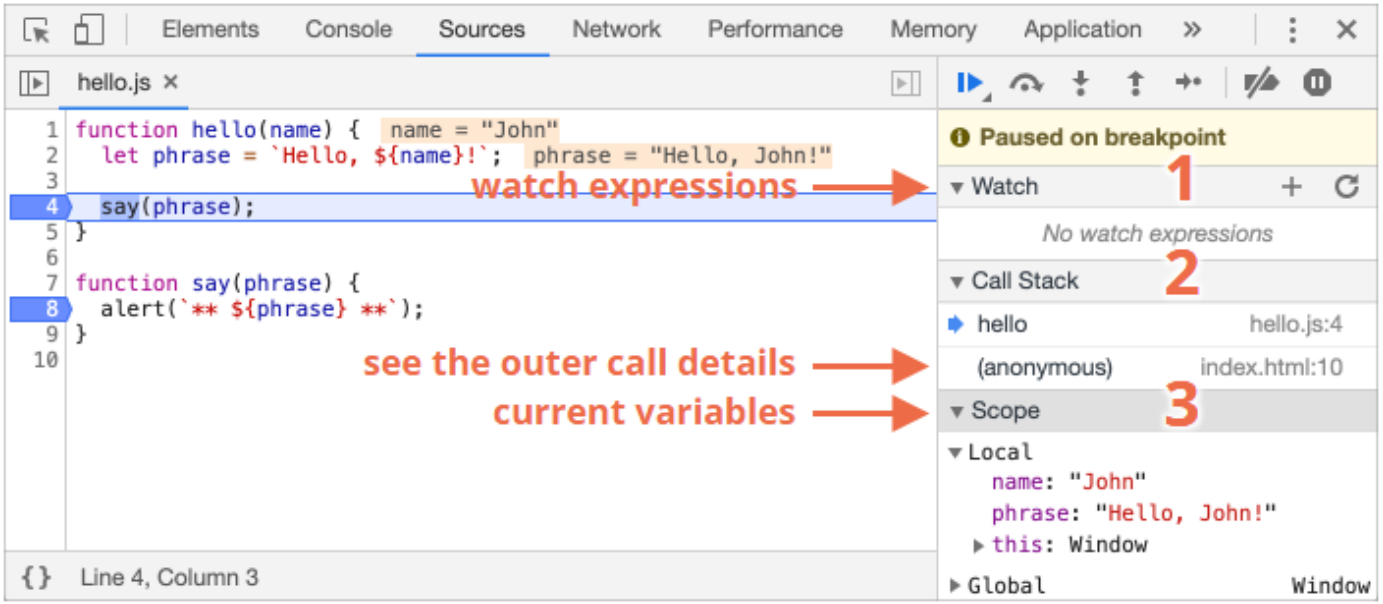
- Please open the informational dropdowns to the right (labelled with arrows). They allow you to examine the current code state:

Pause and Look Around



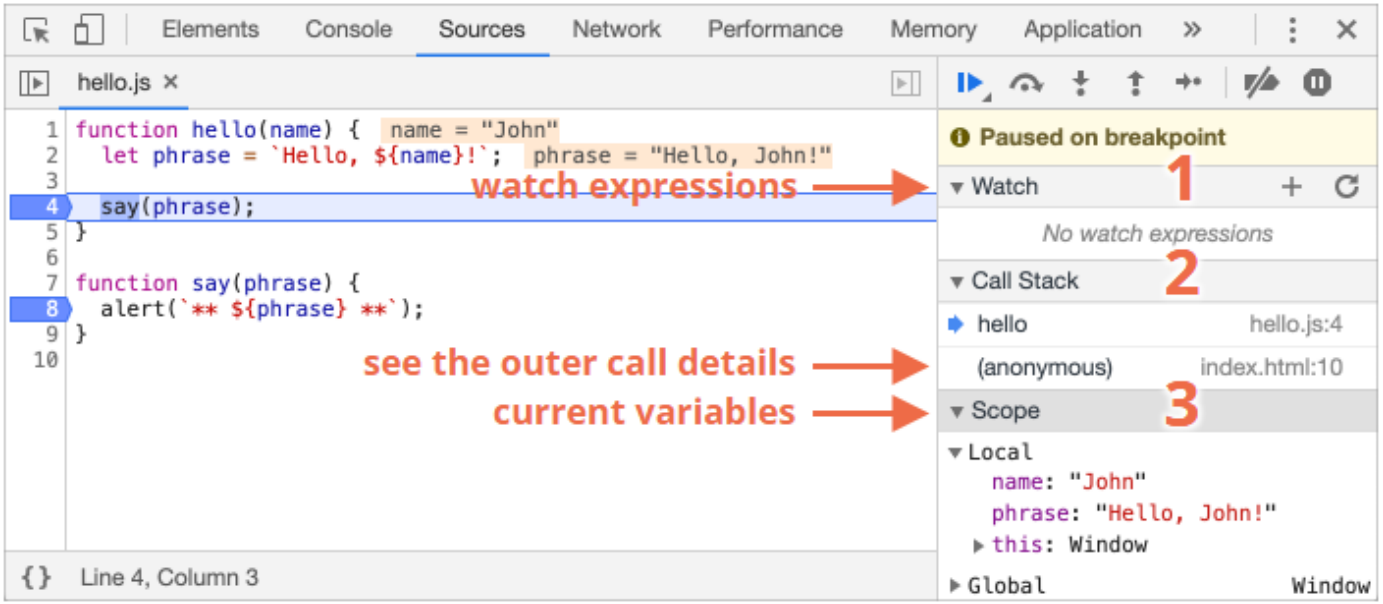
1. Watch – shows current values for any expressions.
   - You can click the plus + and input an expression. The debugger will show its value at any moment, automatically recalculating it in the process of execution.

Pause and Look Around



2. Call Stack – shows the nested calls chain.
   - At the current moment the debugger is inside hello() call, called by a script in index.html (no function there, so it's called "anonymous").
   - If you click on a stack item (e.g. "anonymous"), the debugger jumps to the corresponding code, and all its variables can be examined as well.

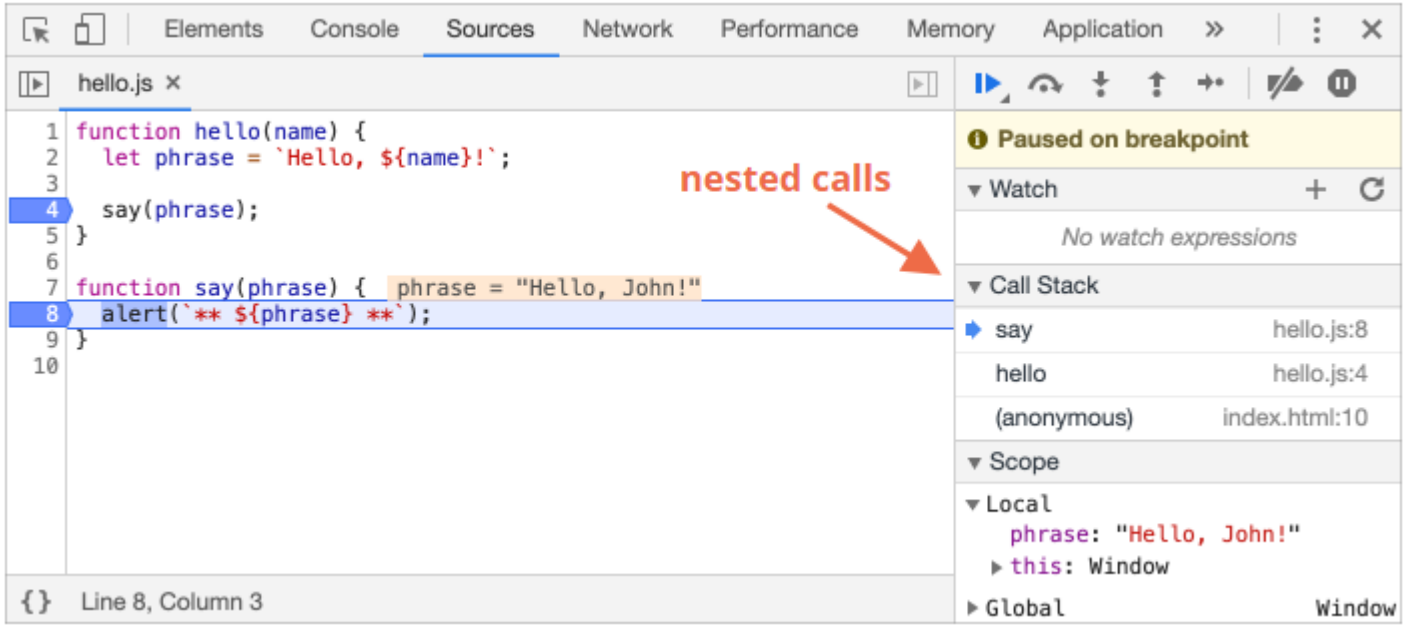
Pause and Look Around



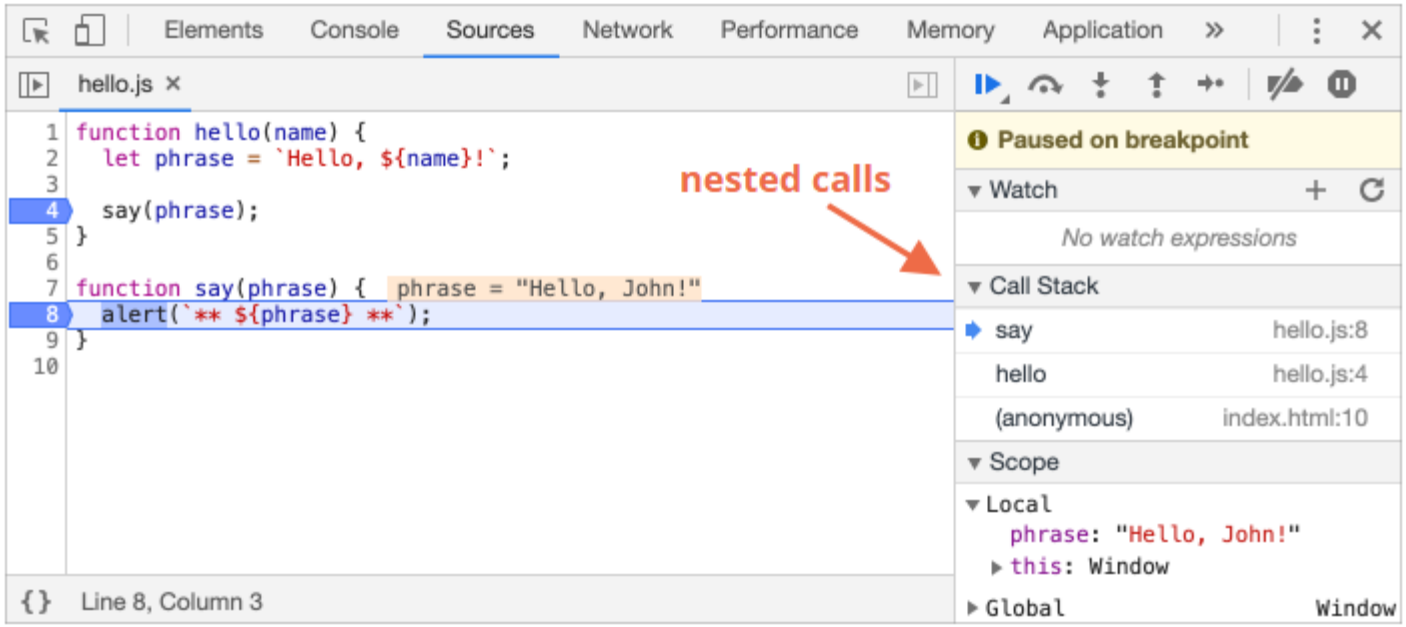
3. Scope – current variables.
   - Local shows local function variables. You can also see their values highlighted right over the source.
   - Global has global variables (out of any functions).

Tracing the Execution

- There are buttons for it at the top of the right panel.

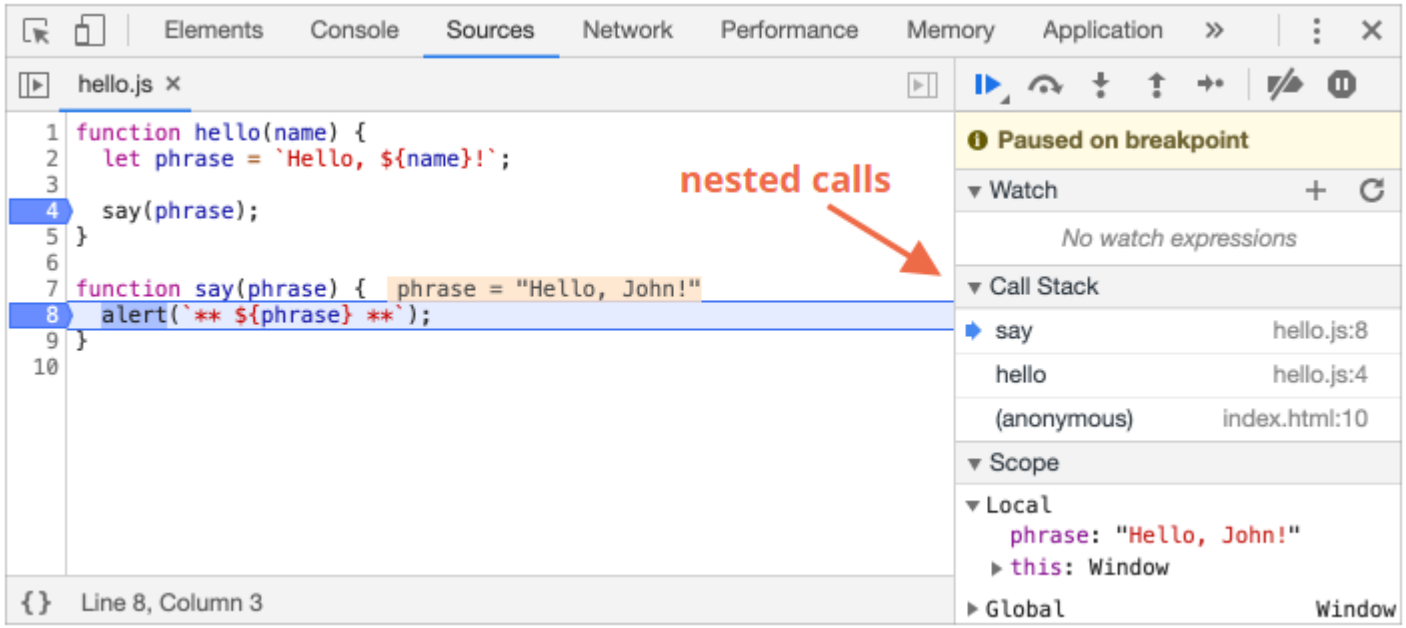


Tracing the Execution



- ▶ – "Resume": continue the execution, hotkey F8.
  - Resumes the execution. If there are no additional breakpoints, then the execution just continues and the debugger loses control.

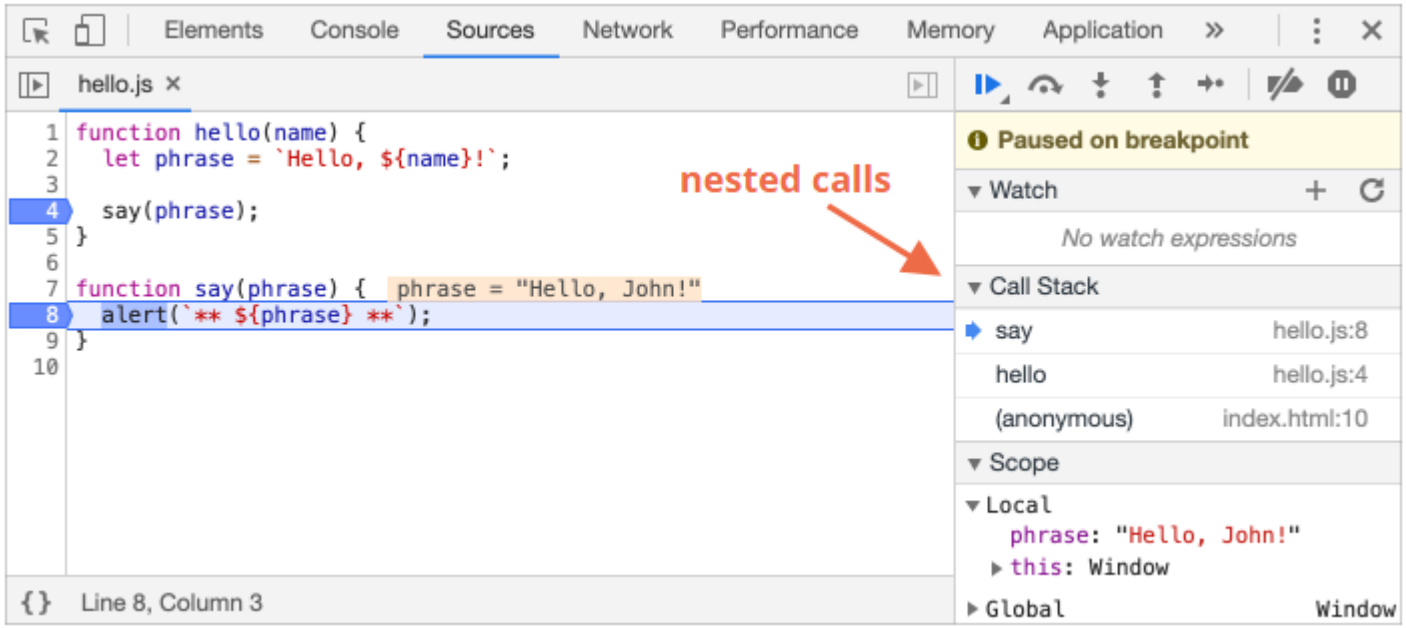
Tracing the Execution



- "Resume": continue the execution, hotkey F8.
  - Resumes the execution. If there are no additional breakpoints, then the execution just continues and the debugger loses control.

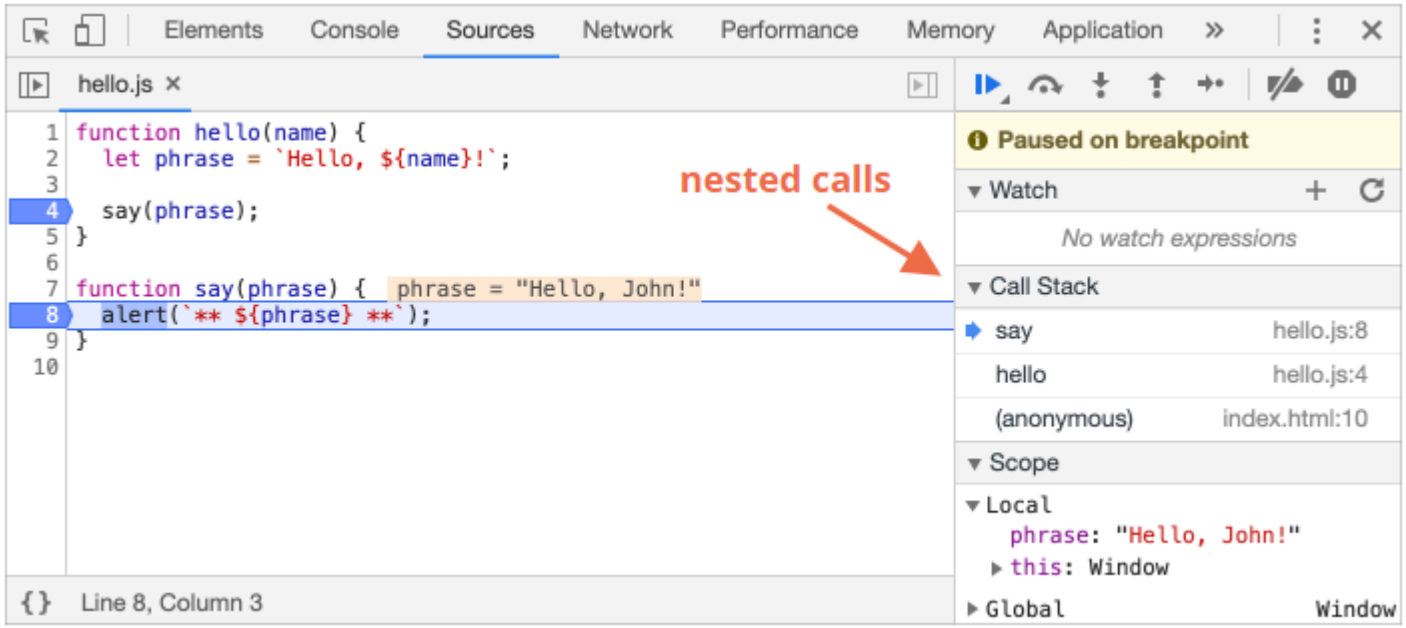
- The execution has resumed, reached another breakpoint inside say() and paused there. Take a look at the "Call Stack" at the right. It has increased by one more call. We're inside say() now.

Tracing the Execution



- "Step": run the next command, hotkey F9.
  - Run the next statement. If we click it now, alert will be shown.
- Clicking this again and again will step through all script statements one by one.

Tracing the Execution

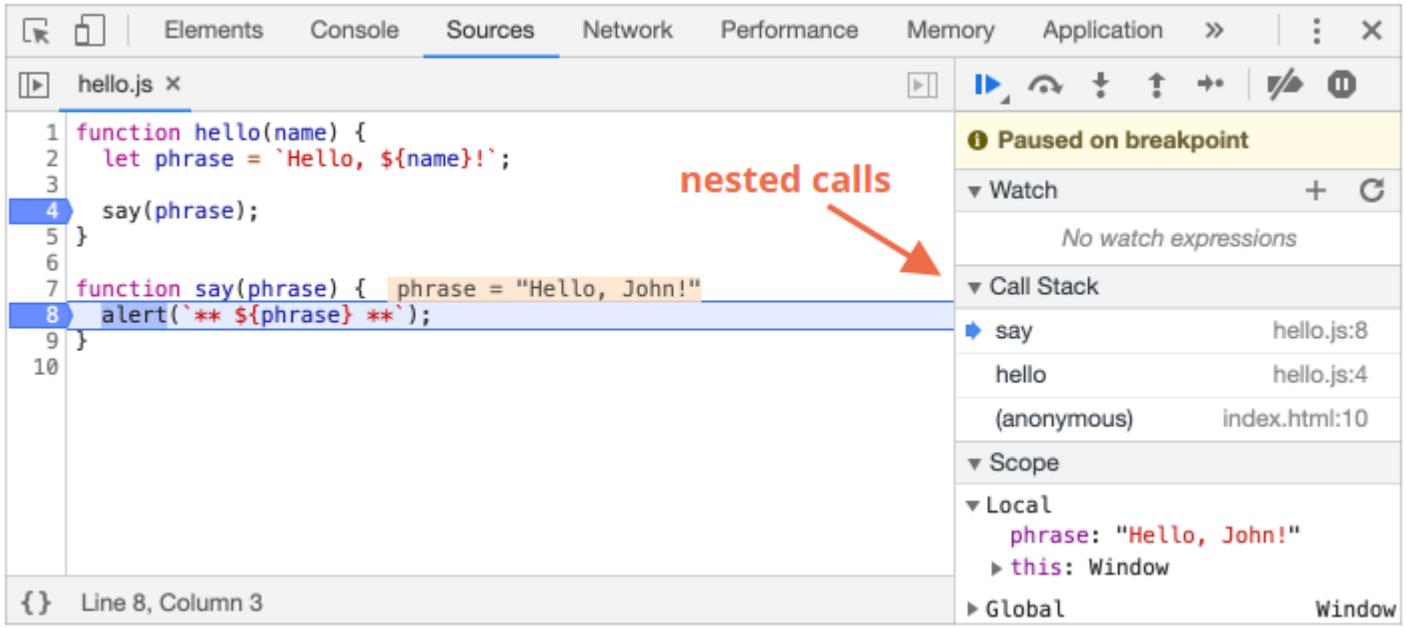


- "Step over": run the next command, but don't go into a function, hotkey F10.
  - Similar to the previous "Step" command, but behaves differently if the next statement is a function call.

- The "Step" command goes into it and pauses the execution at its first line, while "Step over" executes the nested function call invisibly, skipping the function internals.

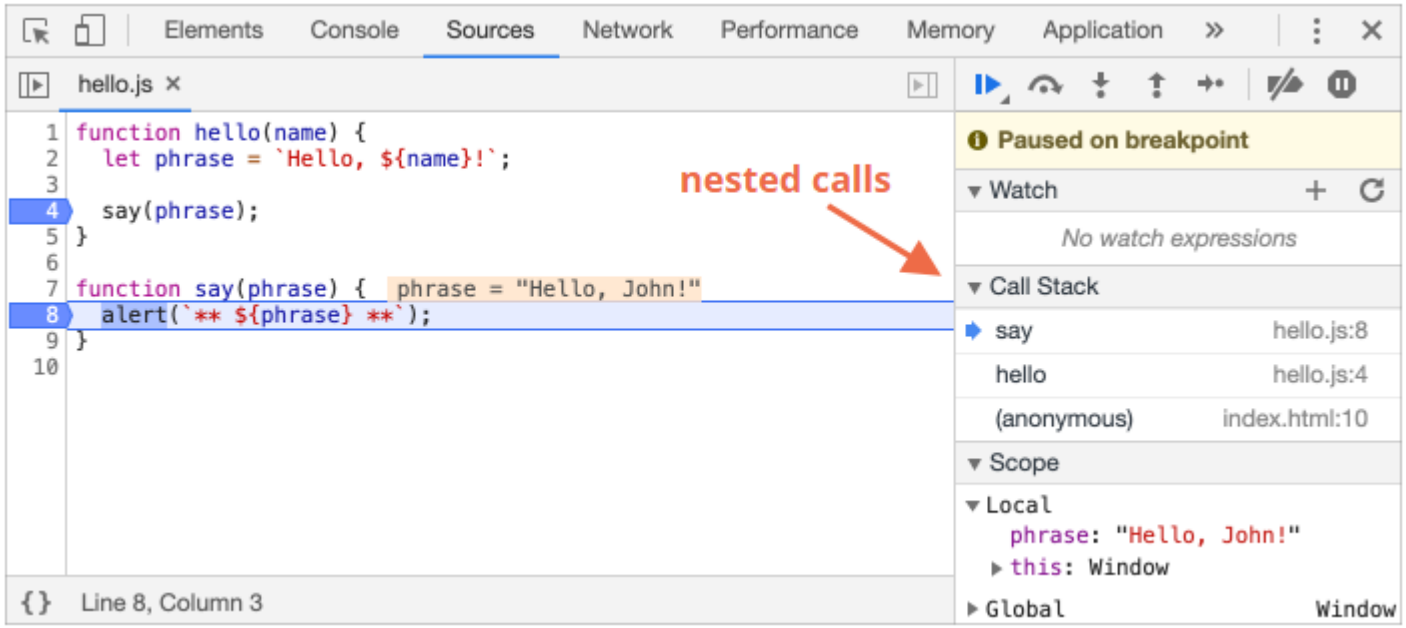
- The execution is then paused immediately after that function.

Tracing the Execution



- "Step into", hotkey F11.
  - That's similar to "Step", but behaves differently in case of asynchronous function calls. If you're only starting to learn JavaScript, then you can ignore the difference, as we don't have asynchronous calls yet.

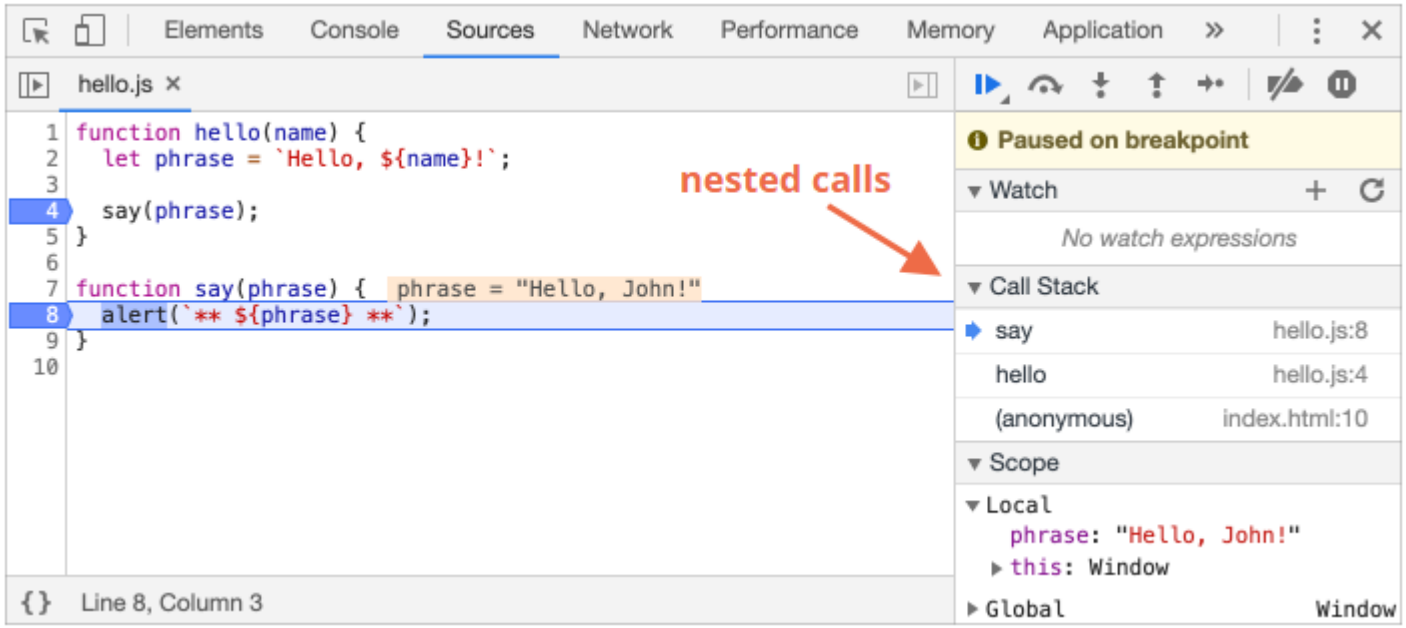
Tracing the Execution



- "Step out": continue the execution till the end of the current function, hotkey Shift+F11.
  - Continue the execution and stop it at the very last line of the current function.

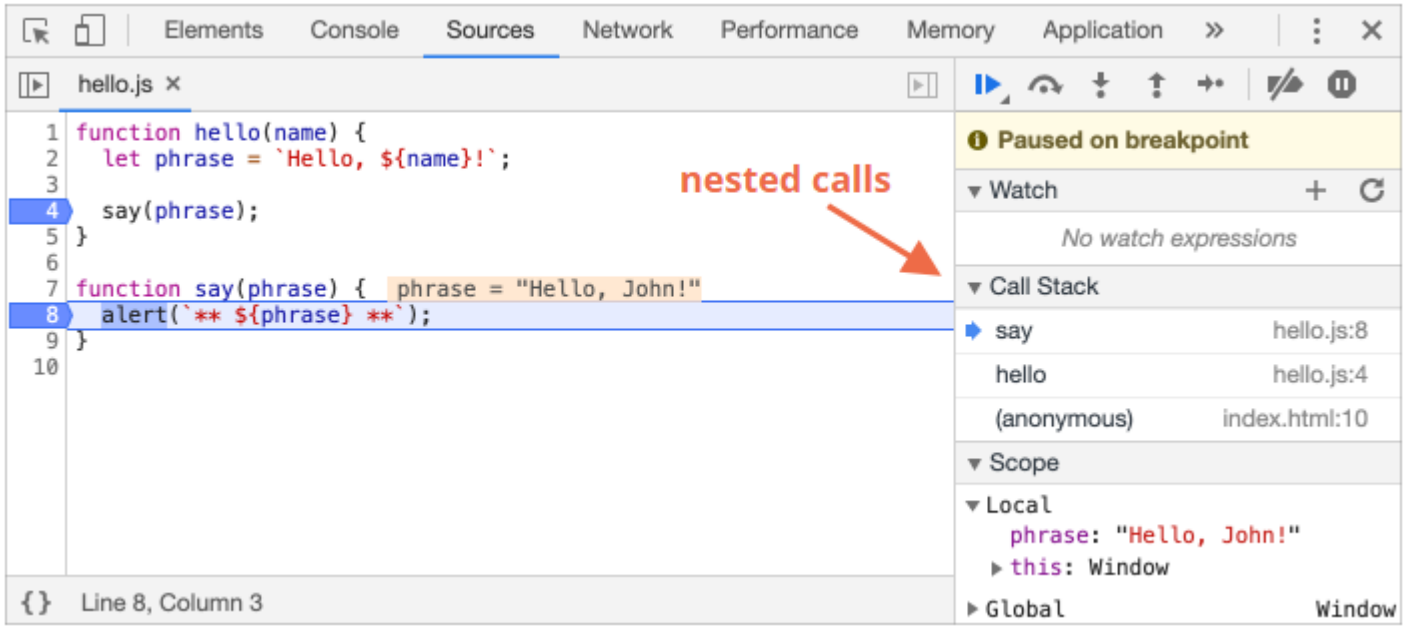
- That's handy when we accidentally entered a nested call using , but it does not interest us, and we want to continue to its end as soon as possible.

Tracing the Execution



- Enable/disable all breakpoints.
  - That button does not move the execution. Just a mass on/off for breakpoints.

Tracing the Execution



- Enable/disable automatic pause in case of an error.
  - When enabled, and the developer tools is open, a script error automatically pauses the execution.
  - Then we can analyse variables to see what went wrong. So if our script dies with an error, we can open debugger, enable this option and reload the page to see where it dies and what's the context at that moment.